



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Aplicaciones con tecnologías de internet
Laboratorio #5



Reto #5

Estudiante:

Edwyn Guzmán

30326271

1. Identifica en tu código el tipo de uso de this

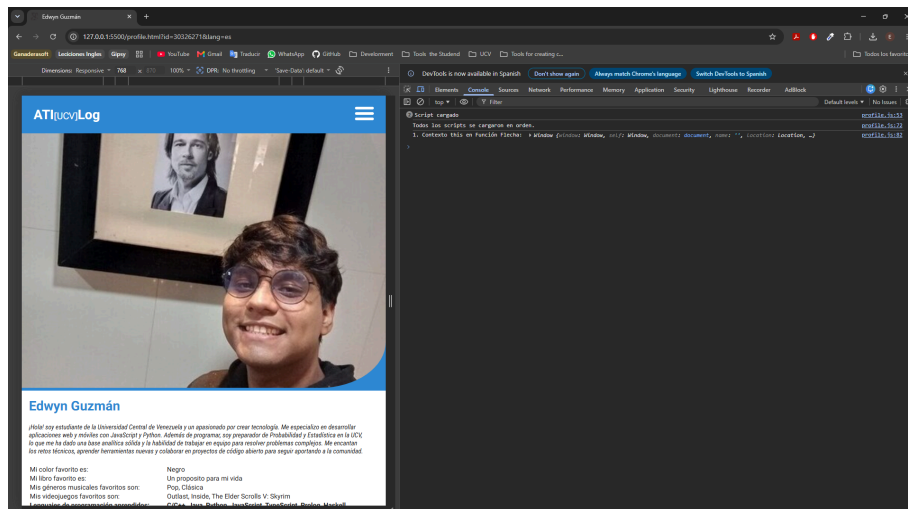
Función 1

En este entorno, la función flexha no posee un contexto this propio, por lo que hereda directamente el valor del ámbito léxico en el que fue definida. Dado que esta función fue declarada en el nivel principal del script, adopta el contexto global, haciendo que this apunte por defecto al objeto window del navegador.

Imagen 1. Código

```
1 // Función para mostrar la información del perfil en la página, recibe una ruta opcional (por defecto es la raíz del proyecto)
2 const show_profile_data = (root = '/') => {
3
4   console.log("1. Contexto this en Función Flecha:", this);
5
6   let profile_img_src_small = root + id + "/" + id + "Small" + profile.image_ext;
7   let profile_img_src_big = root + id + "/" + id + "Big" + profile.image_ext;
8   profile_container.innerHTML = `
9     <div class="profile-container-left">
10       <div>
11         <picture>
12           <source media="(max-width: 768px)" srcset="${profile_img_src_small}" class="profile-img" type="image/jpeg" id="profile-img-small">
13           
14         </picture>
15       </div>
16     </div>
17     <div class="profile-container-right">
18       <h1 class="profile-title" id="profile-title">${profile.name}</h1>
19
20       <p class="profile-description" id="profile-description">
21         ${profile.description}
22       </p>
23
24       <table class="profile-container-feature">
25         <tr class="feature-1">
26           <td id="favorite-color-label">${config.color}</td>
27           <td id="favorite-color-campus">${profile.color}</td>
28         </tr>
29         <tr class="feature-2">
30           <td id="favorite-book-label">${profile.book.length <= 1 ? config.book[0] : config.book[1]}</td>
31           <td id="favorite-book-campus">${profile.book.toString().replaceAll(', ', ' ')}</td>
32         </tr>
33         <tr class="feature-3">
34           <td id="favorite-music-genre-label">${profile.music.length <= 1 ? config.music[0] : config.music[1]}</td>
35           <td id="favorite-music-genre-campus">${profile.music.toString().replaceAll(', ', ' ')}</td>
36         </tr>
37         <tr class="feature-4">
38           <td id="favorite-video-game-label">${profile.video_game.length <= 1 ? config.video_game[0] : config.video_game[1]}</td>
39           <td id="favorite-video-game-campus">${profile.video_game.toString().replaceAll(', ', ' ')}</td>
40         </tr>
41         <tr class="feature-5">
42           <td id="learned-languages-label">${config.language}</td>
43           <td id="learned-languages-campus">${profile.language.toString().replaceAll(', ', ' ')}</td>
44         </tr>
45       </table>
46
47       <div class="contact-info">
48         <p>
49           ${config.email.replaceAll('[email]', '<a href="mailto:${profile.email}">${profile.email}</a>')}
50         </p>
51       </div>
52     </div>
53   `;
54 }
55
56 }
```

Imagen 2. Objeto this



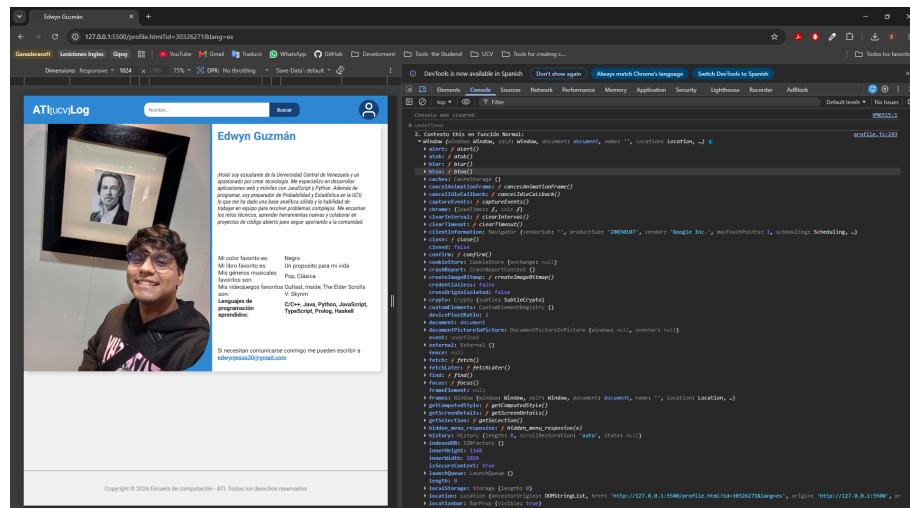
Función 2

En el entorno de una función clásica, el valor de `this` no es fijo, sino que depende estrictamente de la forma en que la función sea invocada. Al ejecutarse de manera regular y sin tener el modo estricto ("use strict") activado en esa sección del código, el contexto recae en el ámbito global, por lo que `this` hace referencia al objeto global `window`.

Imagen 3. Código

```
1 // Logica para ocultar menu responsive al cambiar el tamaño de la pantalla
2 function hidden_menu_responsive(e) {
3     // CAPTURA 2: Función Normal
4     console.log("2. Contexto this en Función Normal:", this);
5
6     if (window.innerWidth > 768) {
7         container_nav_responsive.classList.remove("inset-0");
8         container_nav_responsive.classList.remove("relative");
9     }
10 }
```

Imagen 4. Objeto this



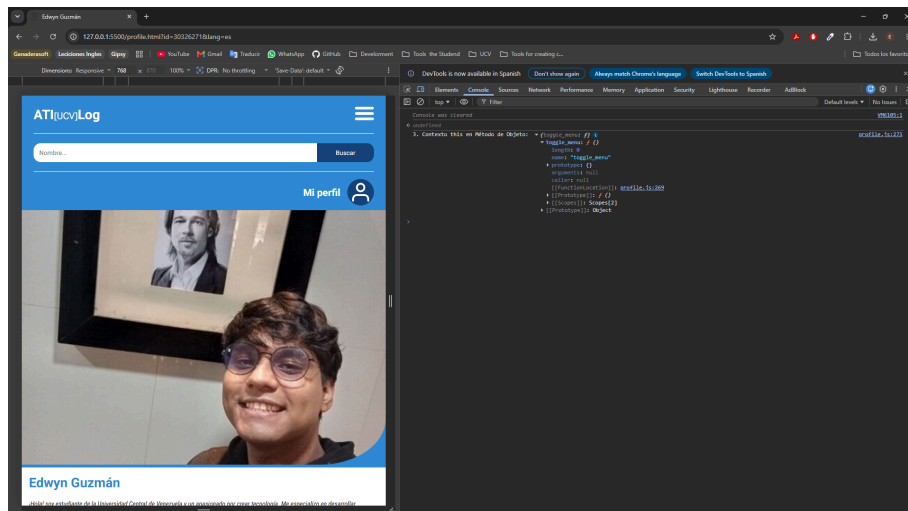
Función 3

Este entorno se genera de manera exclusiva cuando una función se define como una propiedad dentro de la estructura de un objeto. En este tipo de contexto, la palabra reservada `this` apunta de forma directa al propio objeto que está ejecutando el método, o sea, un ambiente local, lo cual en el código corresponde a `menuController`.

Imagen 5. Código

```
1 // Lógica para mostrar menu responsive
2 const menuController = {
3   toggle_menu: function() {
4     // CAPTURA 3: Método de Objeto
5     console.log("3. Contexto this en Método de Objeto:", this);
6
7     container_nav_responsive.classList.toggle("inset-0");
8     container_nav_responsive.classList.toggle("relative");
9   }
10  };
```

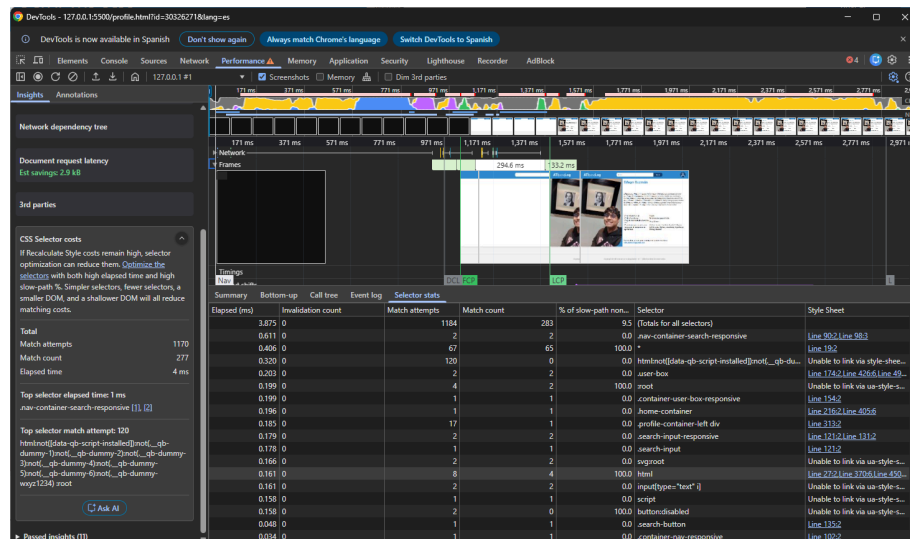
Imagen 6. Código



2. Analice el rendimiento del selector de CSS

Al revisar las estadísticas de DevTools, logramos identificar cuáles son los selectores de CSS que están haciendo trabajar más al navegador. El que más tiempo consume es **.nav-container-search-responsive** (0.611 ms). Aunque es una clase directa y sencilla, tarda un poco más porque seguramente se encarga de calcular estilos más pesados para la versión móvil, como la distribución flexbox o las transiciones. Le sigue el selector universal ***** (0.406 ms), que resulta muy costoso porque obliga al sistema a revisar absolutamente todos los elementos de la página uno por uno (haciendo 67 intentos en total). Por otro lado, selectores generales como **.user-box** y **:root** tienen un consumo de tiempo normal (unos 0.200 ms), ya que se limitan a leer la estructura principal y las variables globales. Sin embargo, el verdadero candidato a optimizar es **.profile-container-left div** (0.185 ms). Este es un problema clásico de rendimiento: al ser un selector descendente, obliga al navegador a recorrer varios niveles del código hasta encontrar el elemento correcto. Esto queda en evidencia al ver que hace 17 intentos de búsqueda solo para encontrar 1 coincidencia, haciendo que la página procese información de más sin necesidad.

Imagen 7. Rendimiento CSS



Optimización

Para aplicar la mejora, vamos a atacar el selector **.profile-container-left div** siguiendo el principio del laboratorio: simplificar la especificidad para evitar que el navegador recorra múltiples niveles del DOM.

Imagen 8. Modificamos el HTML.

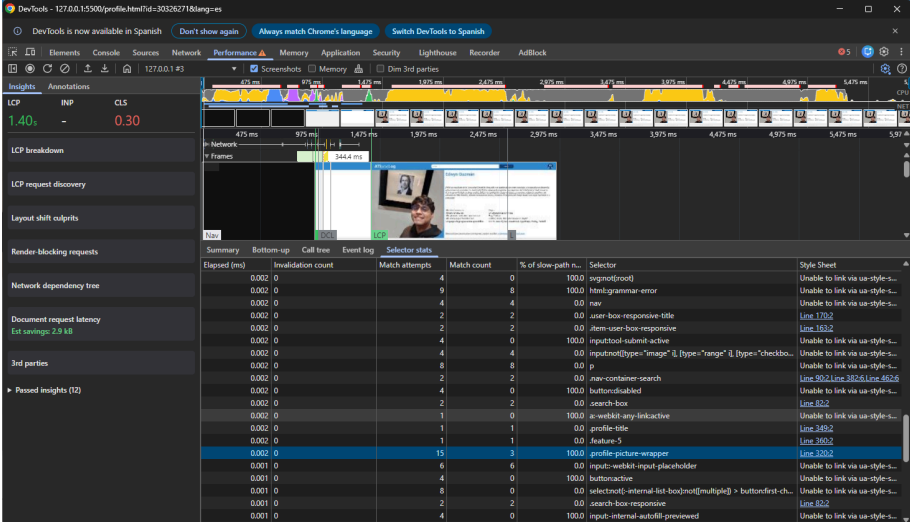
```
1 <div class="profile-container-left">
2   <div class="profile-picture-wrapper">
3     <picture>
4       <source media="(max-width: 768px)" srcset="{profile_img_src_small}" class="profile-img" type="image/jpeg"
5       id="profile-img-small">
6       
7     </picture>
8   </div>
9 </div>
```

Imagen 9. Optimización del CSS

```
1  /* Antes (Costoso) */
2  /* .profile-container-left div {
3      display: flex;
4      height: 100%;
5  } */
6
7  /* Optimización */
8  .profile-picture-wrapper {
9      /* estilos */
10     display: flex;
11     height: 100%;
12 }
```

Esta simplificación reduce la cantidad de elementos procesados en cada actualización, disminuyendo los intentos de coincidencia y acortando el tiempo de ejecución en los eventos Recalculate Style.

Imagen 10. Rendimiento CSS optimizado



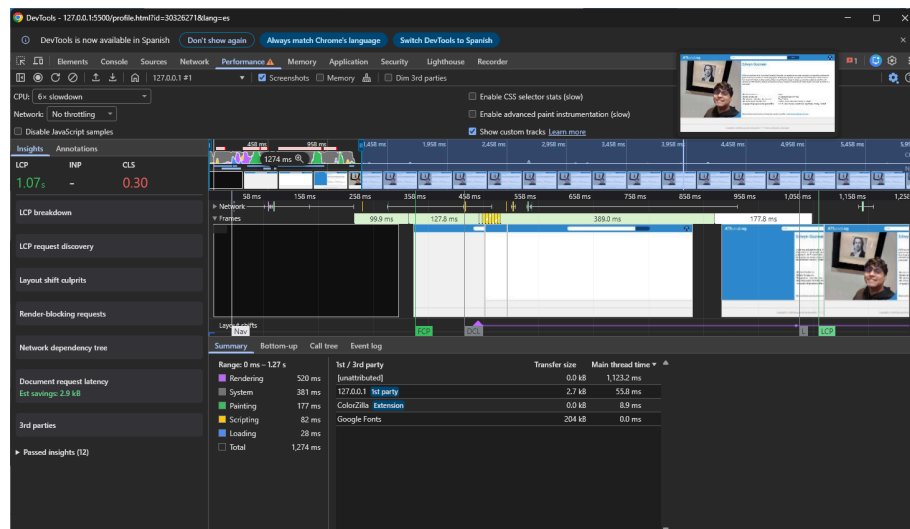
	Summary	Bottom-up	Call tree	Event log	Selector state	Match count	% of slow path n...	Selector	Style Sheet
Render-blocking requests	Elapsed (ms)	Invalidation count			Match attempts				
Network dependency tree	0.002	0			4	0	100.0	svgnotfound	Unable to link via us-style-...
	0.002	0			9	8	100.0	htmlgrammar-error	Unable to link via us-style-...
	0.002	0			4	4	0.0	nav	Unable to link via us-style-...
Document request latency	0.002	0			2	2	0.0	user-box-responsive-fds	Line 1732
Est savings: 2.9 kB	0.002	0			2	2	0.0	user-box-responsive	Line 1632
3rd parties	0.002	0			4	0	100.0	inputtool-submit active	Unable to link via us-style-...
	0.002	0			4	4	0.0	inputnot[type="image" i, type="range" i, type="checkbox...	Unable to link via us-style-...
	0.002	0			2	2	0.0	p	Unable to link via us-style-...
	0.002	0			8	8	0.0	profile-pic	Line 3494
	0.002	0			2	2	0.0	new-container-search	Line 3022 Line 3023 Line 3024
Passed insights (12)	0.002	0			4	0	100.0	buttondisabled	Unable to link via us-style-...
	0.002	0			2	2	0.0	search-box	Line 822
	0.002	0			1	0	100.0	ai-webkit-any-linkactive	Unable to link via us-style-...
	0.002	0			1	1	0.0	profile-pic	Line 3494
	0.002	0			1	1	0.0	feature-5	Line 3802
	0.002	0			15	3	100.0	profile-picture-wrapper	Line 3202
	0.001	0			6	6	0.0	input-webkit-input-placeholder	Unable to link via us-style-...
	0.001	0			4	0	100.0	buttonactive	Unable to link via us-style-...
	0.001	0			8	0	0.0	selectnot-internal-td-bodynot[multiple] > buttonfirst-ch...	Unable to link via us-style-...
	0.001	0			2	2	0.0	search-box-responsive	Line 822
	0.001	0			4	0	100.0	input-internal-autofill-previewed	Unable to link via us-style-...

Se reemplazó el selector descendiente original por una clase directa (**.profile-picture-wrapper**), evitando que el navegador tenga que recorrer múltiples niveles del árbol DOM. Esta simplificación redujo la complejidad de la búsqueda, bajando el tiempo de procesamiento de 0.185 ms a tan solo 0.002 ms. Como resultado, el navegador procesa menos elementos, optimizando los eventos de recálculo de estilo y mejorando el rendimiento general de la página.

3. Analiza la aplicación

En esta prueba limpia con la CPU ralentizada (6x), podemos observar que el código JavaScript es bastante eficiente, consumiendo apenas 82 ms. Sin embargo, el cuello de botella ahora se identifica claramente en las tareas de rendering (520 ms) y painting (177 ms). En la franja superior de la gráfica se aprecian marcadores rojos (Long Tasks) y caídas de FPS justo en la fase de carga de la interfaz. Esto indica que, en un dispositivo de bajos recursos, al navegador le cuesta trabajo recalcular la estructura y dibujar los elementos visuales en pantalla, lo que genera esa falta de fluidez.

Imagen 11. Rendimiento de la aplicación



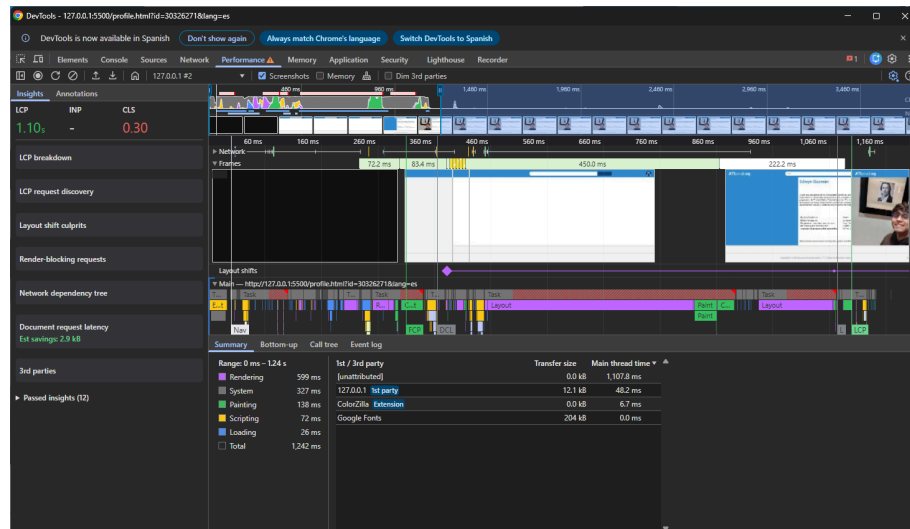
Optimización

Se implementó la propiedad **will-change: transform, opacity**; en los contenedores interactivos para notificar al navegador sobre futuras mutaciones visuales. Esto permite que el motor de renderizado preprocese la animación delegando la carga a la GPU, evitando repintados innecesarios. Como resultado, se reduce el consumo de CPU en las fases de rendering/painting y se garantiza una navegación fluida en dispositivos móviles.

Imagen 12. Optimización

```
1  /* Optimización para animaciones suaves */
2  .container-nav-responsive, .card {
3    will-change: transform, opacity;
4  }
```

Imagen 13. Rendimiento al optimizar



Al contrastar los resultados de esta nueva prueba con el estado inicial, se evidencia una mejora clara y generalizada en el rendimiento. El impacto más notable se observa en la fase de pintado, cuyo tiempo disminuyó de 177 ms a 138 ms. Adicionalmente, el esfuerzo del hilo principal dedicado a la aplicación local se optimizó de 55.8 ms a 48.2 ms, de la mano con una reducción en el procesamiento de JavaScript, que bajó de 82 ms a 72 ms. En conjunto, estas mejoras aliviaron la carga del navegador, permitiendo que el tiempo de ejecución global se redujera de 1,274 ms a 1,242 ms y confirmando la efectividad de la optimización implementada.